📦 Spring IoC Container Summary

The **Inversion of Control (IoC) Container** is the core component of the Spring Framework, responsible for managing the lifecycle and dependencies of application objects, known as **Beans**.

1. What is IoC?

Inversion of Control (IoC) is a principle where the control of object creation, configuration, and management is passed from the programmer's code to the container (the framework).

- **Traditional Approach:** The code manually creates and manages its dependencies (new Repository()).
- **IoC Approach:** The container is responsible for creating and providing the dependencies when they are needed.

2. Dependency Injection (DI)

Dependency Injection (DI) is the mechanism used by the IoC Container to achieve IoC. Instead of a component creating its dependencies, the container "injects" them.

Concept	Description
Bean	Any object instantiated, assembled, and managed by the Spring IoC container.
Wiring	The process of connecting beans together (i.e., injecting one bean into another).
Benefits	Promotes **loose coupling**, improves **testability**, and increases **modularity**.

3. Configuration Metadata

The IoC container needs configuration metadata to know which objects to create and how to wire them together. This is provided through one of three methods:

1. **XML-based configuration** (Older approach).
2. **Java annotations** (Modern, preferred approach, e.g., @Component, @Service).
3. **Java code** (Configuration classes using @Bean).

4. Key Annotations for DI

Annotation	Purpose	Placement
@Component	A generic stereotype for any Spring-managed component.	Class level
@Service	Specialized @Component used for service-layer classes containing business logic.	Class level
@Repository	Specialized @Component used for data access objects (DAOs) interacting with a database.	Class level
@Autowired	Tells Spring to automatically inject a dependency into a field, constructor, or setter method.	Field, Constructor, or Setter

5. Lifecycle of a Bean

The IoC container manages the full lifecycle of a Bean:

1. **Instantiation:** The container creates the bean instance.
2. **Population:** Dependencies are injected (DI).
3. **Initialization:** Lifecycle callbacks (like methods annotated with @PostConstruct) are executed.
4. **Ready:** The bean is ready for use by the application.
5. **Destruction:** The bean is destroyed when the container closes (e.g., methods annotated with @PreDestroy).